

CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

Autonomous Drone Communication Using LoRa and MAVlink for Long-Range Control

A graduate project submitted in partial fulfillment of the requirements

For the degree of Master of Science in
Computer Engineering

By
Andres Flores

December 2024

The graduate project of Andres Flores is approved:

Dr. Jack Ou

Date

Dr. Xiaojun (Ashley) Geng

Date

Dr. Shahnam Mirzaei, Chair

Date

California State University, Northridge

Table of Contents

Signatures.....	ii
List of Figures	v
Lis of Tables	v
Abstract	vi
1. Introduction	1
1.1 Background	1
1.2 Objective	1
2. Design	3
2.1 Hardware	3
2.2 Tools	6
3. Operational Safety and Regulatory Compliance	8
3.1 Safety	8
3.2 Legal Considerations	8
4. Autonomous Drone	9
4.1 MAVlink Protocol	9
4.2 MAVlink Message Format	9
4.3 Circular Fly Path	10
5. LoRa Module	12
6. Implementation	16
7. Performance	19
7.1 Test.....	19
7.2 Result	20
8. Conclusion	22
9. Source Code	23
9.1 Ground Station Python.....	23
9.1.1 Ground_Control.py	23
9.1.2 Ground_Control_LoRa_Commands.py	24
9.2 Onboard Computer System Python	25
9.2.1 Main.py	25
9.2.2 LoRa_Commands.py	32
9.2.3 Connect_to_Vehicle.py.....	34
9.2.4 Set_mode.py.....	34
9.2.5 Check_Pre_Arm.py.....	35
9.2.6 Arm_Drone.py	36
9.2.7 Disarm_Drone.py.....	37
9.2.8 Takeoff.py	37
9.2.9 Set_Movement.py	38
9.2.10 Range_Test.py	44
9.2.11 Get_Location.py.....	47
9.2.12 Drone_Menu.py	49
9.2.13 Lidar_Distance.py.....	49
9.2.14 Land.py	52
9.2.15 Signal_Test.py	53
9.2.16 If_number.py.....	54

9.2.17 Log_File.py	55
10. Work Cited.....	56

List of Figures

Figure 1.1-1: System Block Diagram	3
Figure 1.1-2: Drone Frame	4
Figure 1.1-3: PX4 2.4.8 Flight Controller	4
Figure 1.1-4 Raspberry Pi 4	5
Figure 1.1-5: LoRa Module RLYR998	5
Figure 1.1-6 TF-Luna LiDAR	6
Figure 3.2-1: FAA Maximum Altitudes in San Fernando Valley	8
Figures 4.1-1: Position Targeting in NED Frame	10
Figures 4.1-1: Position Targeting in Global	10
Figures 5.1-1: Bandwidth vs Range	12
Figures 5.1-2: Fresnel Zone	13
Figure 6.1-1: Drone Commands	16
Figure 7.2-1 RSSI vs Distance Graph	21
Figure 7.2-2: Aerial Map of Waypoint in Test Flight.....	21

List of Figures

Table 7.2 -1: Lora Range Test 672 Meters	20
--	----

Abstract

Autonomous Drone Communication Using LoRa and MAVlink for Long-Range Control

By
Andres Flores

Master of Science in Computer Engineering

This project focuses on extending the range of communication from an autonomous drone control by integrating LoRa technology with MAVlink protocol on an onboard computer (Raspberry Pi). This system will increase the range from traditional communication protocols like Wi-Fi or Bluetooth where range has limitations. Key objectives include establishing LoRa communication where commands can be sent or received between ground stations and the onboard computer. The command received by the onboard computer will be able to translate into a MAVlink message for the pixhawk flight controller. Testing performance in various environments demonstrates significant range improvement making this approach viable and low power for long-range autonomous missions.

1. Introduction

1.1. Background

In recent years, unmanned aerial vehicles (UAVs) commonly known as drones had an increase in demand, driven by their application in agriculture, surveillance, emergency management, etc. UAVs have enhanced operation efficiency and data collection capabilities. As these technologies evolve, enhancing communication capabilities remains a crucial challenge as long-range communication has become crucial.

Traditional communication methods, such as Wi-Fi and Bluetooth, present limitations when used for communication with the onboard computer of a UAV. While these technologies can work effectively over short distances, they often struggle to maintain stable connections over longer ranges or in environments with significant interference, such as urban settings or remote areas with rugged terrain. Consequently, there is a pressing need for more robust and reliable communication solutions that can ensure continuous connectivity, even when the UAV operates far from its ground station.

Long Range (LoRa) is a modulation technique based on spread spectrum technology, specifically derived from chirp spread spectrum (CSS) technology^[4]. It offers several advantages that make it particularly suited for UAV applications. One of the most significant benefits of using LoRa is its ability to facilitate low-cost, low-power communication while operating in unlicensed frequency bands. This characteristic allows it to transmit small packets of data over remarkably long distances

1.2. Objective

This project is to implement LoRa technology to establish seamless communication between the onboard computer of the unmanned aerial vehicle (UAV) and the ground station. This innovative communication framework is designed to enhance the operational capabilities of the UAV, allowing it to send telemetry messages that enable real-time monitoring of its status and performance. The integration of LoRa technology not only facilitates the transmission of

telemetry data but also supports the bidirectional flow of critical mission commands from the ground station to the UAV. These commands can encompass a wide range of operational instructions, including flight paths, waypoint navigation, altitude adjustments, and other essential directives that ensure the UAV can execute its missions autonomously and effectively.

A key advantage of utilizing LoRa technology in this project is its impressive long-range capabilities, which ensure reliable communication over extended distances. This is particularly valuable in environments such as open fields and suburban areas, where traditional communication methods may falter due to signal degradation or interference. By leveraging LoRa's low-power, long-range transmission capabilities, the project aims to create a robust communication network that maintains a consistent connection between the UAV and ground station, even in challenging conditions.

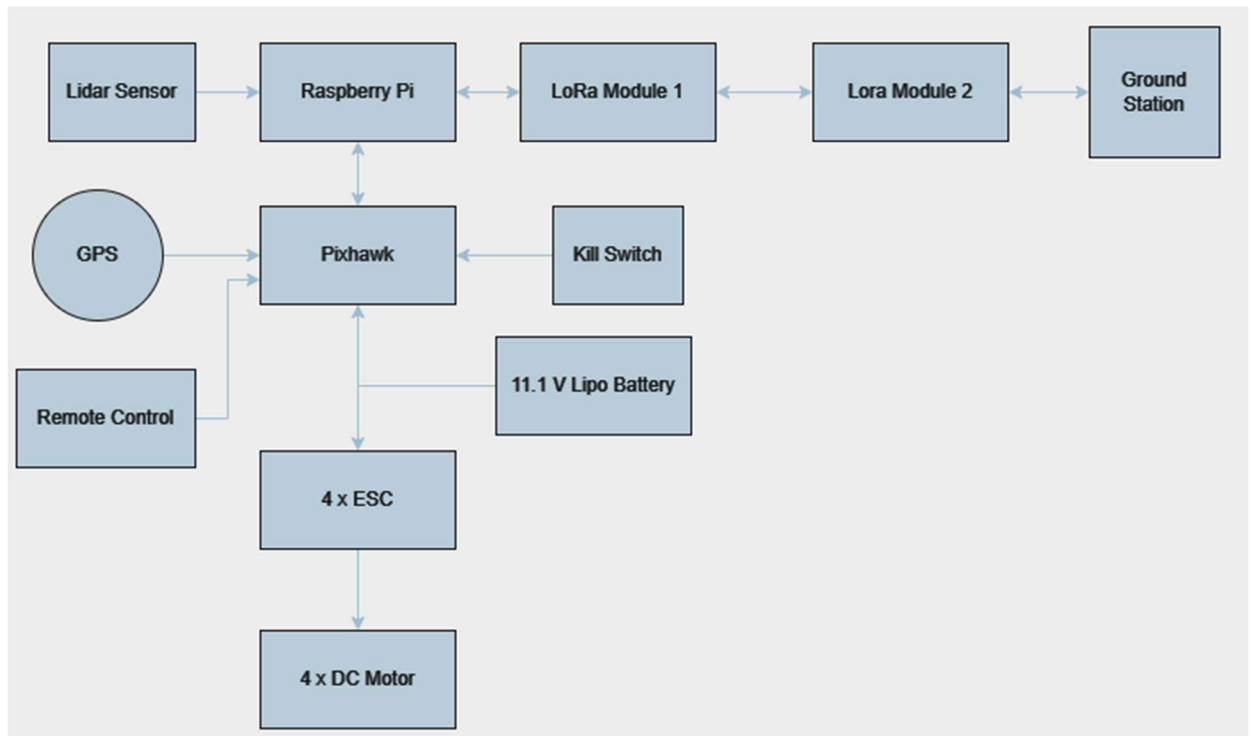
The successful implementation of LoRa technology will ultimately enable autonomous flights with a significantly greater communication range than is typically achievable with standard radio or Wi-Fi systems. This extended range is crucial for various applications, including agricultural monitoring, search and rescue operations, and environmental surveying, where the UAV may need to operate far from the ground station. Moreover, the ability to send and receive mission commands in real-time ensures that operators can adjust flight parameters dynamically, enhancing the UAVs adaptability to changing mission requirements and environmental conditions.

2. Design

2.1. Hardware

The drone system utilizes a range of integrated hardware components to enable autonomous flight and long-range communication capabilities. Figure 1.1-1 illustrates the system diagram, showing how all the modules are interconnected to create a functional autonomous drone system.

Figure 1.1-1 System Block Diagram



The Hawks' Work F450 Drone Kit was one of the best options for hobbyists and professionals for building custom drones as shown in figure 1. The F450 frame typically has a diagonal size of about 450mm, enough space to add components. The frame is usually made of lightweight yet durable materials, such as carbon fiber or plastic, which helps in maintaining flight stability while keeping the overall weight low.

Figure 1.1-2: Drone Frame



At the heart of the F450 drone is the Pixhawk PX4 2.4.8 Flight Controller, a powerful and flexible system designed for a wide range of UAV applications. The Pixhawk supports GPS integration, which enables precise navigation and waypoint execution. The flight controller features two telemetry ports for connecting devices that facilitate real-time data transmission to the onboard computer. Also, include the motors output to connect to eight brushless motors.

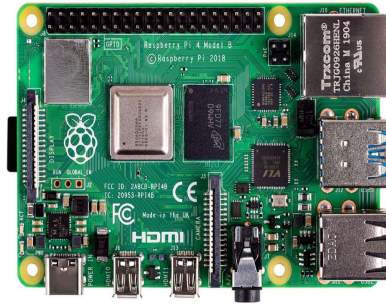
Figure 1.1-3: PX4 2.4.8 Flight Controller



I utilized the Raspberry Pi 4 as the onboard computer, a powerful and versatile platform ideal for handling the complex tasks required for autonomous flight. The Raspberry Pi communicates with the Pixhawk flight controller via serial communication (UART), establishing a reliable connection that facilitates seamless data transfer and command execution. Raspberry Pi serves as the central hub for processing sensor data. It collects data from the Pixhawk and other connected sensors, analyzes it, and extracts meaningful insights to guide the flight operations. It

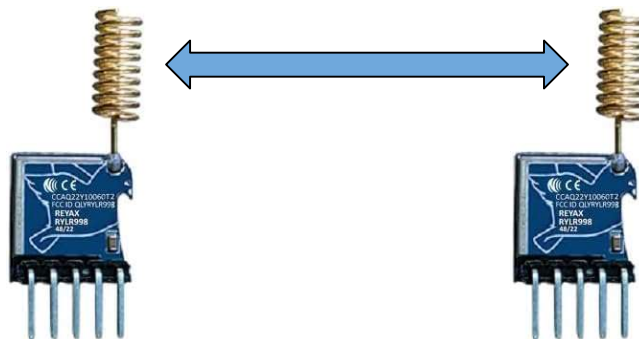
can send and receive MAVlink commands through the serial link, which establishes a robust communication protocol between the Raspberry Pi and the Pixhawk.

Figure 1.1-4: Raspberry Pi 4



For the LoRa module, I chose the RLYR998, a reliable UART interface operating at 868/915 MHz, known for its robust long-range, low-power communication capabilities. In this project, I am using two RLYR998 modules to facilitate bidirectional communication between the drone and the ground station, each serving a specific purpose. One module is connected to the onboard computer (a Raspberry Pi) on the drone, where it interfaces with the flight controller and the various sensors to receive telemetry data and send command instructions as needed. The other module is connected to a PC, which functions as the ground station. This setup allows the PC to act as a remote control and monitoring hub, receiving real-time flight data and enabling manual intervention or adjustments if necessary.

Figure 1.1-5: LoRa Module RLYR998



By integrating the TF-Luna LiDAR range sensor, the drone will be capable of accurately measuring object distances up to eight meters. This functionality enhances safety by enabling

effective obstacle avoidance, ensuring the drone can navigate its environment with precision and reduce the risk of collisions.

Figure 1.1-6 TF-Luna LiDAR



Other components used for the F450 were a GPS module, (4) ESCs, (4) brushless motors, safety switch, Remote Control and 11.1V LiPo Battery. Overall, this setup integrates the Raspberry Pi and LoRa module for long range control while the pixhawk manages the flight on F450 frame.

2.2 Tools

Mission Planner was used to ensure that each component in the drone setup works together seamlessly, serving as the primary ground station software for configuration and calibration. This open-source application supports vital setup tasks for key components like the GPS, compass, accelerometer, electronic speed controllers (ESCs), and the remote control (RC). Calibration of the compass within Mission Planner prevents magnetic interference, which can disrupt heading accuracy and cause instability during flight. Mission Planner also supports accelerometer calibration, allowing the flight controller to accurately interpret tilt and acceleration, which is essential for maintaining stability and correctly understanding movement. Also, tools for calibrating the ESCs, which synchronize motor speeds and allow for balanced lift and smooth flight performance. Without ESC calibration, the motors may not respond evenly, leading to unstable flight.

Python plays a versatile role in handling various aspects of the system, from communication to automation. Python is my primary tool for scripting tasks, processing data,

and implementing protocols. By leveraging Python's flexibility, I was able to integrate complex communication protocols, such as MAVlink, with the LoRa module on the onboard computer, enabling the drone to send and receive telemetry data over long distances. This setup is key to ensuring real-time mission control and monitoring, even when the drone is far from the ground station.

3. Operational Safety and Regulatory Compliance

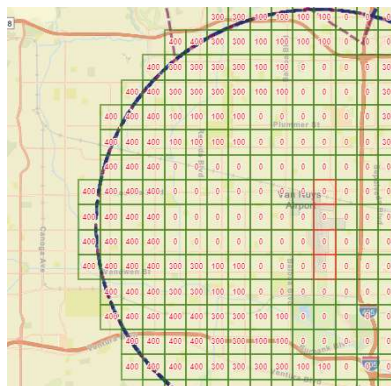
3.1 Safety

The remote control serves as a backup for unforeseen circumstances. It provides three key options: Force Land, Manual Control, and Kill switch. In the event of an onboard computer failure, the *Force Land* function ensures an emergency landing. The Manual Control option allows the operator to regain control and guide the drone to a safe landing if the onboard system is unresponsive but poses no immediate risk. In critical situations where the drone endangers others, the Kill switch immediately cuts power to the motors, ensuring safety.

3.2 Legal Considerations

The Federal Aviation Administration (FAA) is a division of the U.S Department of Transportation that is responsible for civil aviation and commercial space transportation in the United States. To legally operate any aircraft, including drones, in shared airspace, compliance with FAA rules and regulations is mandatory. These regulations cover a wide range of operational standards, such as pilot certification requirements, maximum altitude limits, restricted no-fly zones, and specific conditions for commercial versus recreational use. Operators must adhere to these guidelines to minimize risks to people, property, and other aircraft, as well as to protect national security interests.

Figure 3.2-1: FAA Maximum Altitudes in San Fernando Valley



4. Autonomous Drone

4.1 MAVlink Protocol

MAVLink (Micro Air Vehicle Link) is a lightweight messaging protocol specifically designed for drones and other unmanned systems. It uses a predefined message format to transmit various types of data, including telemetry information and control signals. Ground station software such as Mission Planner, or custom applications, leverage MAVLink to upload flight plans, monitor flight paths, and execute commands like takeoff, landing, and hovering. Additionally, MAVLink plays a crucial role in real-time telemetry, providing operators with vital data to monitor drone health and performance.

For the flight controller to accept control signals via MAVLink, the drone must first be set to Guided Mode. This mode enables autonomous operation by allowing the onboard computer to send and execute commands, ensuring precise and flexible control during missions.

4.2 MAVlink Message Format

In autonomous drone flight, there are two commands that are utilized for movement in the horizontal plane x and y axis: “SET_POSITION_TARGET_LOCAL_NED” and “SET_POSITON_TARGET_GLOBAL_INT”.^[1] A bitmask is used to indicate using X, Y and Z position only and ignore the velocity, acceleration and yaw/yaw rate field. The SET_POSITION_TARGET_LOCAL_NED is a MAVlink command that uses the target position for the drone in the local coordinate system. The command will be sent with value to move forward/North, backward/South, right/EAST or left/West when giving a distance value to travel. There are two visions of this command when the frame value is either set at “MAV_FRAME_LOCAL_NED” or “MAV_FRAME_BODY_OFFSET_NED”. The key difference is that MAV_FRAME_LOCAL_NED uses the current position; if x is given a positive value, it will travel North and MAV_FRAME_BODY_OFFSET_NED uses current position and heading so giving x a positive value it will travel forward where the drone is currently facing. The SET_POSITON_TARGET_GLOBAL_INT command won’t accept distance values in the x and y axis instead it will accept latitude and longitude values. In this

message the altitude parameter can be set relative to different references to home position, terrain or sea level, for my project I use relative to home.

Figure 4.1-1: Position Targeting in NED Frame ^[1]

```
master.mav.send(mavutil.MAVlink.MAVlink_set_position_target_local_ned_message (
    10, master.target_system,
    Master.target_component,

mavutil.MAVlink.MAV_FRAME_BODY_OFFSET_NED,
    int (0b110111111000),
    x, y, -z
    0, 0, 0,
    0, 0, 0,
    0, 0))
```

Figure 4.1-2: Position Targeting in Global ^[1]

```
master.mav.send(mavutil.MAVlink.MAVlink_set_position_target_global_int_message (
    10, master.target_system,
    Master.target_component,

mavutil.MAVlink.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    int(0b110111111000),
    int(lat*1e7), int (lon* 1e7), ALT,
    0, 0, 0,
    0, 0, 0,
    0, 0))
```

4.2 Circular Fly Path

One of my autonomous fly paths on the drone is the circular fly path. However, there is no predefined MAVlink command message that will make the drone fly in a circular trajectory. The solution I came up with was to calculate eight waypoints arranged in a circular formation. The first step is to obtain the current position. Then use the formulas to obtain the new latitude and longitude, where d is set to 6371000 the earth radius and R equals to the desired radius of the circular flight path:

$$lat_{new} = lat_{current} + \frac{d}{R} \times \sin(\theta) \quad (1)$$

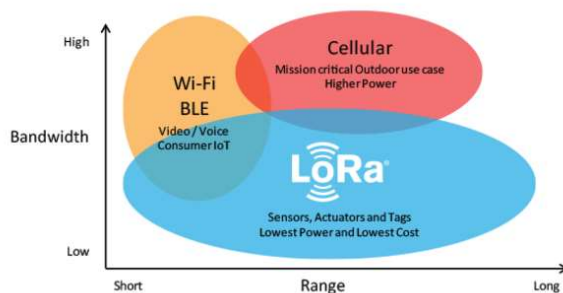
$$lon_{new} = lon_{current} + \frac{d}{R} \times \frac{\sin(\theta)}{\cos(lat_{current})} \quad (2)$$

Implement these calculations within a for loop that will iterate eight times, increasing the angle by 45 degrees with each iteration.

5. LoRa Module

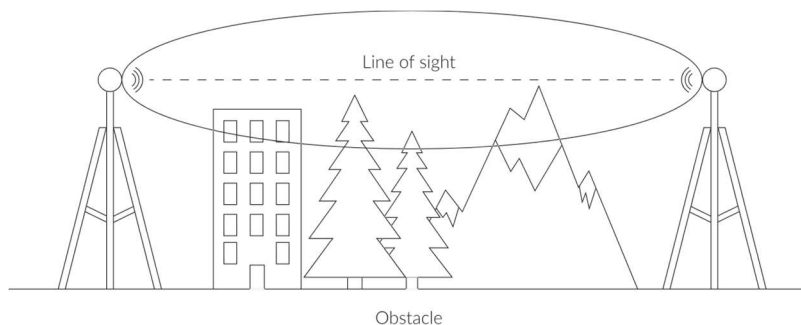
LoRa (Long Range) is a wireless communication technology designed for low-power, long-range communication, ideal for IoT applications. LoRa wireless uses spread spectrum modulation, which spreads a narrowband signal over a wider channel bandwidth.^[6] This technique allows the signal to be more resilient to interference and ensures reliable communication over long distances. LoRa's long-range capabilities come from several clever optimizations, including Chirp Spread Spectrum (CSS) modulation, where the frequency of the signal increases or decreases over time (a chirp).^[4] This modulation method enables LoRa to maintain reliable communication while consuming minimal power. This makes LoRa highly suitable for remote sensing. One of the key advantages of LoRa is its ability to maintain long-range communication with low power consumption, allowing devices to run on batteries for several years. LoRa operates in the unlicensed ISM (Industrial, Scientific and medical) radio band across the world. Each country has their own frequency set and in the United States we use frequency of 902-928 MHz.

Figure 5.1-1- Bandwidth vs Range ^[4]



To achieve the best signal quality, the two LoRa modules should have a clear line of sight with no obstacles in between them. The Fresnel zone, however, is an ellipsoid-shaped region that exists between the transmitter and receiver. If any object obstructs this zone, even if it's not directly in the line of sight, it can still impact the signal strength, leading to potential interference and reduced communication range. Maintaining a clear Fresnel zone is crucial for optimal performance and reliable transmission.

Figure 5.1-2 Fresnel Zone



LoRa modules often offer multiple operational modes to balance functionality and power efficiency, and these modes can be configured to suit specific application requirements. Mode 0 (Transceiver mode) is the default mode, where the module operates as a standard transmitter and receiver, enabling real-time communication with no additional power-saving features. This mode is ideal for applications requiring continuous communication and is particularly suitable when real-time updates and control are needed, such as in drone operations where immediate telemetry data and commands are critical. Mode 1 (Sleep mode) is designed for maximum power conservation, where the module enters a low-power state to extend battery life. In this mode, the module does not actively transmit or receive, making it suitable for devices that only occasionally need to wake up and communicate. Lastly, Mode 2 (Smart receiving power-saving mode) provides a middle ground by alternating between receiving mode and a low-speed mode, which reduces power consumption while maintaining some level of communication readiness. However, in this mode, careful adjustment of transmission timing is necessary to ensure compatibility with the module's operational intervals. These configurable modes offer flexibility to optimize the LoRa module's performance for specific use cases, whether prioritizing responsiveness or power efficiency. For my drone, I use Mode 0 to achieve real-time updates and control, ensuring seamless communication for immediate telemetry feedback and mission commands.

The LoRa RYLR998 is a long-range transceiver module that is designed for low-power wireless communication applications. The module can receive and transmit messages by using Attention (AT) commands. AT commands are a set of instructions used to communicate between

cellular or RF modems and host controllers. The AT commands were created by Dennis Heyes as the Hayes Command Set for the ‘Hayes 200 Baud Smart Model’ in 1981 [5]. The format of the command will always start with “AT”, followed by specific instructions. For example, “AT+ADDRESS?” the response will be “ADDRESS=120”. This command is asking the module what address is assigned to.

The LoRa module has parameters that need to be set-up at start-up Network ID, Address, Bandwidth. To communicate between two modules, they need to be set-up on the same network ID range from 3-18. Each module will be assigned to a unique address ranging from 0-65535, 0 is default. The bandwidth has three options 125KHz (default), 250 kHz and 500 kHz, where 125KHz is the smaller bandwidth but better sensitivity more suitable for long distance but longer transmission time.

To send data with AT command format, the command is structured as “AT+SEND = <Address>, <Payload Length>, <DATA> “ [3]. The <Address> specifies the module address to which the data is being sent, while the <Payload Length> indicates the length of the string. If we want to send the string “Hello World” the payload length will be 11 as space is counted.

When the onboard computer sends a message, the <Data> package will contain a tag that indicates the type of message being sent, such as “INPUT”, “TEST”, or “MENU”. Each tag serves a specific purpose for the ground station in displaying the message to the operator.

- INPUT: This tag informs the ground station that it requires input from the operator. The operator will need to respond to the message
- TEST: When the ground station receives this tag, it indicates that an automatic confirmation message should be sent back, acknowledging receipt data
- MENU/INFO: This tag simply displays the message on the operator's screen without requiring any action or response.

To read data it will be received in the format of “+RCV = <Address>, <Length>, <DATA>, <RSSI>, <SNR>”. The <Address> represents sender’s address, <Length> is the

length of the data, <DATA> is the received message in ASCII, <RSSI> refers to the received signal strength indicator and <SNR > represents the signal-to-noise ratio. The RSSI ranges from 0 to -127 where values closer to 0 indicate a stronger signal.

6. Implementation

Upon connecting the onboard computer to power, the script will automatically execute. The LoRa module is assigned an address “1” for the onboard computer, while the ground station address will be “2”. The ground station will initially wait for a connection message, prompting the operator with “Connect to Flight Controller y/n”. Once the connection between the onboard computer and the flight controller is established. The next command sent by the onboard computer is “Arm Drone y/n”. The script will terminate if ‘n’ is sent, otherwise the drone will proceed to arm then take-off to the predefined altitude. A separate thread will be dedicated to running the script for the LiDAR sensor. This approach ensures that the function responsible for reading distance values operates independently. The decision to use a separate thread was made because the initial readings from the sensor are often inaccurate. By continuously running the LiDAR script in the background, the main thread can access accurate and updated distance values as needed without being disrupted by initialization inaccuracies.

Once the drone is at the target altitude the onboard computer will present a menu of options:

Figure 6.1-1: Drone Commands

1 = Fly Movement
2 = Waypoint
3 = Hover
4 = Circle
5 = Return Home
6 = Return Home and Land
7 = Land
8 = LoRa Com Test

The operator can select a command by sending the corresponding number. If option 1 “Fly Movement” is selected, the ground station will receive the message “X-axis Value,” asking how far the drone should move forward or backward based on the input: a positive value for

forward, a negative value for backward, and zero for no movement along that axis. After receiving the X-axis value, the drone will then prompt for the “Y-axis Value,” which will determine the movement to the right or left in the same manner. Once both values are received, the drone will execute the specified movements.

Option 2, “Waypoint”, the system will prompt the operator to input the target coordinates. First, the ground station will receive the message “Enter Latitude: “where it allows the operator to input desired latitude value. After receiving a valid latitude value, the system will follow up with “Enter Longitude: “once it receives both values the onboard computer will process these coordinates and send the command to the flight controller. The drone will then fly autonomously to the specified coordinates and upon arrival at the waypoint, the drone will hover and maintain its position at the current position.

Option 3, “Hover”, the system will prompt the operator “Enter Altitude: “, where the operator can enter a new target altitude for the drone. Upon receiving the input, the drone can adjust its altitude accordingly by increasing/decrease. This adjustment overrides the predefined altitude value set during the initial takeoff. It is important to note that this command specifically enables the operator to change the drone's altitude in real time.

Option 4, “Circle”, the system will help operators to set up the circular flight path for the drone. First, the ground station will receive the message “Enter Radius”, where its operator will input the desired radius circular path. Once the onboard computer receives a valid input it will send the ground station “Clockwise/ Counter-Close wise 0/1: “. This will choose the direction of the circular flight ‘0’ for clockwise direction and ‘1’ for a counterclockwise direction. Once both inputs are received, the onboard computer will compute eight waypoints based on the current position and radius value then the drone will autonomously fly to all eight waypoints, maintaining the specified radius and direction while flying at the current altitude.

Option 5, “Return Home” the onboard computer will use the home latitude and longitude value that was automatically recorded before the drone was armed. These values correspond to the drone’s takeoff point, ensuring that the drone can autonomously navigate back to its original

location. It will fly autonomously to the position and once the drone reaches its home position, it will hover at its current altitude, awaiting further commands. This command is useful when we want the drone to come back to safe space but remain airborne for future instructions.

Option 6 “Return Home and Land” the process is very similar to option 5. The onboard computer will again guide the drone back to the home position. Instead of hovering at the home position, the drone will automatically initiate its landing sequence where it descends smoothly and disarm its motor once it is on land and terminate the script. This option is particularly useful when the mission is complete, and the operator wants the drone to return home and safely land without any further interaction.

Option 7, “Land,” the system commands the drone to land immediately at its current position. Unlike the return home options, the drone will not attempt to fly back to its takeoff point but will instead descend directly from wherever it is in the air. Once the drone has landed, it will disarm its motors and shut down the system, finishing the script. This command can be called if there is a safety issue and needs an emergency landing or the drone needs to land in the current position.

During flight, if the LiDAR sensor detects an object within two meters, the drone will automatically stop and fly backward to maintain a safe distance. Simultaneously, it will send a warning signal to the operator to alert them of the potential obstacle. After ensuring the drone is in a safe position, it will send the “Drone Commands” and reevaluate the next step in its flight path.

I have implemented a fail-safe mechanism where the two LoRa modules continuously communicate with each other during flight. If communication is lost for more than five consecutive cycles, the system detects disruption and assumes communication failure. In such cases, the drone will automatically initiate a return-to-home procedure and safely land along its flight path.

7. Performance

7.1 Test

In the previous section, we explained the commands available in the drone menu. The last option, number 8, is called "LoRa Com Test." This option allows the operator to test the LoRa signal based on a specified distance. When the operator selects option 8, the ground station prompts, "Enter Distance Range:", where the operator can input the distance in meters for testing the LoRa signal range.

The test begins by dividing the entered "Distance Range" into five waypoints. For instance, if the operator inputs 100 meters, the onboard computer will create five waypoints, each 20 meters apart. These waypoints are calculated using the formula provided in selection 3.

Once all five waypoints are set, the drone will fly to the first waypoint. Upon arrival, it will send a message to the ground station with the "AT" command tagged as "TEST: LoRa CT." The tag "TEST:" will let the ground station know to send an automatic confirmation message indicating successful receipt. The drone will proceed to the next waypoint only if the onboard computer receives this confirmation. It will attempt to send the message up to five times before marking the test as a "Fail".

When the test is marked as "Pass," the drone will initiate a return-to-home sequence, hovering at the specified altitude. It will then display the drone menu and await the operator's selection of the next command. In the event of a failure, the drone will automatically initiate a return-to-home sequence and land safely. Potential reasons for failure may include the ground station being out of range, environmental interference, or disconnection of the ground station's LoRa module. By implementing these fail-safes, the system ensures that the drone can respond appropriately to communication issues, prioritizing safety and reliability in operation.

7.2 Result

The current test was conducted in an open field, with a distance range set to 672 meters. The following table summarizes the results of the test, indicating the Received Signal Strength Indicator (RSSI) at each waypoint:

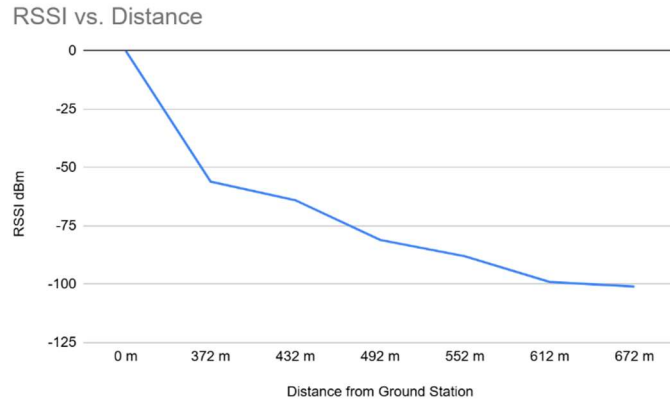
Table 7.2-1: LoRa Range Test 672 Meters

Waypoint	Distance from Ground Station	RSSI
Home	372 m	-56
1	432 m	-64
2	492 m	-81
3	552 m	-88
4	612 m	-99
5	672 m	-101

As observed, the RSSI values decreased with increasing distance from the ground station, indicating the expected attenuation of the signal. The RSSI value ranged from -56 dBm at 372 meters to -101 at 672 meters. However, as the distance increases, the rate of decrease slows down, as observed between the farther waypoints (e.g., between 612 meters at -99 dBm and 672 meters at -101 dBm, a difference of only 2 dBm over 60 meters). In Figure 7.2-2 show the aerial map of the waypoints in the test flight.

The Received Signal Strength Indicator (RSSI) values for LoRa communication typically range from 0 dBm where indicate a very strong signal to -127 dBm where it indicates a very weak signal. According to the data I collected, the figure below shows a chart “RSSI vs Distance”.

Figure 7.2-1: RSSI vs Distance Graph



While the maximum tested distance is currently 672 meters, the system experiences a significant loss of signal strength of approximately 80% over this range. The most pronounced decrease during my testing occurs within the first 60 meters, after the reduction becomes less significant. According to the chart, the signal behaves according to the logarithmic function $f(x) = \log(\frac{l}{x})$, indicating that as distance increases, the rate of signal degradation diminishes.

Figure 7.2-2: Aerial Map of Waypoint in Test Flight



8. Conclusion

The project demonstrates the integration of LoRa technology with MAVlink protocol to extend the communication range of autonomous drones. Using the LoRa RYLR998 module provides a significant advantage over traditional Wi-Fi and Bluetooth systems, and it is complete with satellite communication because LoRa has low power consumption.

From the test it shows the connectivity over extended distances enabled the autonomous drone to be suitable for various applications such as agricultural monitoring, search and rescue operations, and environmental surveying. The implementation of MAVlink commands allowed dynamic control of the drone's flight paths, ensuring adaptability to changing mission requirements.

Future work aimed at extending the range of LoRa modules will focus on developing a mesh network by utilizing multiple LoRa nodes. This innovative approach will significantly enhance the communication capabilities of autonomous drones, ensuring continuous connectivity even when a single drone is out of range from the ground station. By deploying a network of interconnected LoRa nodes, drones can relay messages among themselves, effectively overcoming the limitations of traditional point-to-point communication. Each drone within the mesh network acts as both a transmitter and a receiver, creating a dynamic communication system where data can be shared over greater distances without relying solely on direct links to the ground station.

9. Source Code

9.1 Ground Station Python

9.1.1 Ground_Control.py

```
""" .....
-- Engineer: Andres Flores
-- Description: Main for ground control
..... """
import serial
import time
import msvcrt
from Ground_Control_LoRa_commands import send_command, set_address, get_address,
read_commdnd, set_para

serial_port = 'COM11'
baud_rate = 115200 # Default baud rate for RYLR998
ser = serial.Serial(serial_port, baud_rate, timeout=1)

response = set_address(ser, '2')
print(response)
response = get_address(ser)
print("Address: " + response)
set_para(ser,8,7,1,12)

while True:
    LoRa_rx, rssi, snr = read_commdnd(ser)

    part = LoRa_rx.split(":")
    message_tag = part[0]
    message = part[1]

    if (message_tag == "INPUT"):
        print(message + " [Signal Level:" + rssi + " " + snr + " ]" )
        print("Enter Input: ")
        end_time = time.time() + 10
        while time.time() < end_time: #wait for 10 secound for input
            if msvcrt.kbhit():
                user_input = input()
                send_command(ser, 1, user_input)
                break
```

```

elif(message_tag == "TEST" ):
    send_command(ser, 1, "True")
elif(message_tag == "MENU" or message_tag == "INFO"):
    print(message)
else:
    print(message + " [Signal Level: " + rssi + "]")

```

9.1.2 Ground_Control_LoRa_Commands.py

```

""" .....
-- Engineer: Andres Flores
-- Description: Lora Command functions to be
-- able to read, send and set-up for ground control
..... """
import time
def set_para(ser,SF, BW, CR, Power ):
    command = f'AT+PARAMETER={SF},{BW},{CR},{Power}'
    ser.write((command + '\r\n').encode())
    time.sleep(0.5)
    ser.write(('AT+PARAMETER?' + '\r\n').encode())
    time.sleep(0.5)
    response = ser.read_all().decode()
    print(response)

def send_command(ser, ADDRESS, command):
    command_len = str(len(command))
    c = ('AT+SEND=' + str(ADDRESS) + ',' + command_len + ',' + command + '\r\n')
    ser.write(('AT+SEND=' + str(ADDRESS) + ',' + command_len + ',' + command +
'\r\n').encode())
    time.sleep(0.5) # Wait for the command to be processed
def set_address(ser, address):
    ser.write(('AT+ADDRESS=' + address + '\r\n').encode()) # Send the AT command to query
the address
    time.sleep(0.5) # Wait for the response
    response = ser.read_all().decode() # Read and decode the response
    if response.startswith('+'): # Check if the response is valid
        parts = response.split('=') # Split the response at the '=' character
        return parts[0] # Return the address (after '='), stripped of any whitespace
def get_address(ser):

```

```

ser.write(('AT+ADDRESS?\r\n').encode()) # Send the AT command to query the address
time.sleep(0.5) # Wait for the response
response = ser.read_all().decode() # Read and decode the response
if response.startswith('+'): # Check if the response is valid
    parts = response.split('=') # Split the response at the '=' character
    return parts[1] # Return the address (after '='), stripped of any whitespace
def read_commnd(ser):
    while True:
        string = ser.readline()
        c = string.decode("utf-8")

        if( c != ""):
            if c.startswith('+RCV='):
                parts = c.split(',')
                sender_id = parts[0].split('=')[1]
                message_length = parts[1]
                message_payload = parts[2]
                rssi = parts[3]
                snr = parts[4].strip()
                return message_payload, rssi, snr

```

9.2 Onboard Computer System Python

9.2.1 Main.py

```

""" .....
-- Engineer: Andres Flores
-- Description: The main script facilitates the connection between
-- a ground control system and a drone, enabling commands for
-- autonomous navigation, flight patterns, and telemetry exchange
-- via LoRa communication
..... """
from pymavlink import mavutil
import serial
import time
import sys
import logging
from datetime import datetime
from connect_to_vehicle import connect_to_vehicle
from arm_drone import arm_drone
from disarm_drone import disarm_drone

```

```

from Lora_commands import send_command, get_address, get_network,
read_command, set_parameter
from takeoff import takeoff
from set_movement import fly_movement, fly_to_waypoint, increase_alt, fly_circle
from get_location import get_location
from if_number import is_number_float, is_number_int
from drone_menu import send_drone_flypath_menu
from check_pre_arm import check_pre_arm
from land import land
from Range_Test import test_lora_comm_range
from log_file import setup_log_file
from lidar_distance import get_distance, get_current_distance
from threading import Thread, Event
from Signal_Test import start_thread_signal_connection, stop_signal_connection_thread

setup_log_file()

stop_event = Event()
serial_port = '/dev/ttyUSB1'
baud_rate = 115200 # Default baud rate for RYLR998
GC_Address = 2
altitude = 1.5 # default altitude
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
ser = serial.Serial(serial_port, baud_rate, timeout=1)
distance_thread = Thread(target=get_distance)
distance_thread.start()

try:
    ADDRESS = get_address(ser)
    print("The ADDRESS is:", ADDRESS)

    network = get_network(ser)
    print("The network is:", network)

    set_parameter(ser, 8, 7, 1, 12)

while True: # Establish communication between Drone and Ground Control
    send_command(ser, GC_Address, "INPUT:Connection y/n")
    response = read_command(ser)

```

```

logging.info("Connect to Drone?")
print(response)

if response == 'y':
    logging.info("Yes")
    break
elif response == 'n':
    logging.info("No System OFF")
    sys.exit()
else:
    send_command(ser, GC_Address, "ERROR:Invalid input")

master = connect_to_vehicle()

if master:
    send_command(ser, GC_Address, "INFO:Vehicle Connected")

while True: #ask to arm the drone
    send_command(ser, GC_Address, "INPUT:Arm Drone y/n: ")
    arm_response = read_command(ser)
    logging.info("Arm Drone?")
    if arm_response == 'y':
        logging.info("Yes")
        break
    elif arm_response == 'n':
        logging.info("No System OFF")
        sys.exit()
    else:
        send_command(ser, GC_Address, "INFO:Invalid input")
if check_pre_arm(master):
    home_lat, home_lon, home_alt = get_location(master)
    logging.info("Home Position: %f, %f, %f" % (home_lat, home_lon, home_alt))
    signal_thread = start_thread_signal_connection(master, ser, home_lat, home_lon,
GC_Address, altitude)
    arm_drone(master)
    takeoff(master, altitude)
    stop_signal_connection_thread()
    logging.info("Drone is Arm")
    send_command(ser, GC_Address, "INFO:Drone is armed!")

```

```

try:
    while True:

        drone_command = send_drone_flypath_menu(ser,GC_Address)
        while not is_number_int(drone_command):
            drone_command = send_drone_flypath_menu(ser,GC_Address)

        match int(drone_command):
            case 1: #fly foward in meters
                logging.info("Fly Movement")
                send_command(ser, GC_Address, "INPUT:Enter X axis distance")
                x = read_command(ser)
                if (x == "cancel"):
                    continue
                while not is_number_float(x):
                    send_command(ser, GC_Address, "INPUT:Enter valid x Distance value")
                    x = read_command(ser)
                x = float(x)

                send_command(ser, GC_Address, "INPUT:Enter y axis distance")
                y = read_command(ser)
                if (y == "cancel"):
                    continue
                while not is_number_float(y):
                    send_command(ser, GC_Address, "INPUT:Enter valid y Distance value")
                    y = read_command(ser)
                y = float(y)
                signal_thread = start_thread_singal_connection(master, ser, home_lat, home_lon,
GC_Address, altitude)
                fly_movment(master, x, y)
                stop_signal_connection_thread()
                send_command(ser, GC_Address, "ACK:Has reach to the target distance")

            case 2: #fly to a waypoint
                try:

                    logging.info("Fly to a Waypoint")
                    send_command(ser, GC_Address, "INPUT:Enter Latitude: ")

```

```

        waypoint_lat = read_command.ser)
    if (waypoint_lat == "cancel"):
        continue
    while not is_number_float(waypoint_lat):
        send_command.ser, GC_Address, "INPUT:Enter Latitude: ")
        waypoint_lat = read_command.ser)
    waypoint_lat = float(waypoint_lat)

    send_command.ser, GC_Address, "INPUT:Enter Longitude: ")
    waypoint_lon = read_command.ser)
    while not is_number_float(waypoint_lon):
        send_command.ser, GC_Address, "INPUT:Enter Longitude: ")
        waypoint_lon = read_command.ser)
    waypoint_lon = float(waypoint_lon)

    fly_to_waypoint(master, waypoint_lat, waypoint_lon, altitude )
except Exception as e:
    logging.error("Fly to a Waypoint ERROR: %s", str(e), exc_info=True)
    send_command.ser, GC_Address, "INFO:Fly to a Waypoint ERROR")

case 3: #Hover
    logging.info("Hover")

    send_command.ser, GC_Address, "INPUT:Enter atitude: ")
    ALT = read_command.ser)
    if (ALT == "cancel"):
        break
    while not is_number_float(ALT):
        send_command.ser, GC_Address, "INPUT:Enter atitude: ")
        ALT = read_command.ser)
    ALT = float(ALT)
    ALT_Change = ALT - altitude
    altitude = ALT
    increse_alt(master, ALT_Change)
    time.sleep(2)

case 4: #circle mode
    logging.info("Circle Mode")
    send_command.ser, GC_Address, "INPUT:Enter Radius: ")
    Radius = read_command.ser)

```

```

        if (Radius == "cancel"):
            continue
        while not is_number_float(Radius):
            send_command(ser, GC_Address, "INPUT:Enter Radius: ")
            Radius = read_command(ser)
        Radius = float(Radius)

        send_command(ser, GC_Address, "INPUT:Enter 1 for Clock, 0 for Counter-
Clock")

        dir = read_command(ser)
        if (dir == "cancel"):
            continue
        while not is_number_int(dir):
            send_command(ser, GC_Address, "INPUT:Enter valid direction value")
            dir = read_command(ser)
        dir = int(dir)
        signal_thread = start_thread_signal_connection(master, ser, home_lat, home_lon,
GC_Address, altitude)
        fly_circle(master, Radius, altitude, dir)
        stop_signal_connection_thread()
    case 5: #return home
        logging.info("Return Home")

        fly_to_waypoint(master, home_lat, home_lon, altitude )

    case 6: #return home and land
        try:
            logging.info("Return Home and Land")

            fly_to_waypoint(master, home_lat, home_lon, altitude )
            time.sleep(5)
            land(master)
            break
        except Exception as e:
            logging.error("Return Home ERROR: %s", str(e), exc_info=True)
            send_command(ser, GC_Address, "INFO:Return Home ERROR")

    case 7: #land and break
        logging.info("Land")
        land(master)

```

```

        break

    case 8:
        try:

            logging.info("Testing Commication Range")
            send_command(ser, GC_Address, "INPUT:Enter Distance Range: ")
            Target_distance = read_command(ser)
            if (Target_distance == "cancel"):
                continue
            while not is_number_float(Target_distance):
                send_command(ser, GC_Address, "INPUT:Enter Distance Range: ")
                Target_distance = read_command(ser)
            Target_distance = float(Target_distance)

            pass_test = test_lora_comm_range(master, ser, GC_Address,
            Target_distance,altitude,home_lat, home_lon)
            if (pass_test == False):
                logging.info("LoRa Range has max out")
                fly_to_waypoint(master, home_lat, home_lon, altitude )
                land(master)
                break

        except Exception as e:
            logging.error("TEST Commication RANGE ERROR: %s", str(e),
            exc_info=True)
            send_command(ser, GC_Address, "INFO:TEST Commication RANGE
            ERROR")
            land(master)
            disarm_drone(master)

    case _: #error input
        send_command(ser, GC_Address,"INFO:Invalid input")
        disarm_drone(master)
        send_command(ser, GC_Address, "INFO:Drone is disarmed!")
    except Exception as e:
        logging.error("Drone Menu ERROR: %s", str(e), exc_info=True)
        send_command(ser, GC_Address, "INFO:Drone Menu ERROR")

```

```

except KeyboardInterrupt:
    logging.info("User On Rasberry pi Cancel")
    send_command(ser, GC_Address, "INFO:User On Rasberry pi Cancel")
finally:
    logging.info("Script Finish")
    send_command(ser, GC_Address, "INFO:Finish")

    stop_event.set()
    distance_thread.join()
    ser.close()
    logging.shutdown()
    sys.exit()

```

9.2.2 Lora_commands.py

```

""" .....
-- Engineer: Andres Flores
-- Description: interfacing with a LoRa module using AT
-- commands via a serial connection. It is designed for
-- managing communication parameters, sending and receiving
-- commands, and configuring LoRa settings
..... """
import serial
import time
import logging

def send_command(ser, ADDRESS, command):
    try:
        command_len = str(len(command))
        c = ('AT+SEND=' + str(ADDRESS) + ',' + command_len + ',' + command + '\r\n')
        ser.write(('AT+SEND=' + str(ADDRESS) + ',' + command_len + ',' + command +
\r\n').encode())
        time.sleep(0.5) # Wait for the command to be processed
    except Exception as e:
        logging.info("Error Send command")

def get_address(ser):
    ser.write(('AT+ADDRESS?\r\n').encode()) # Send the AT command to query the address
    time.sleep(0.5) # Wait for the response
    response = ser.read_all().decode() # Read and decode the response
    if response.startswith('+'): # Check if the response is valid
        parts = response.split('=') # Split the response at the '=' character

```

```

        return parts[1].strip() # Return the address (after '='), stripped of any whitespace
def get_network(ser):
    ser.write(('AT+NETWORKID?\r\n').encode())
    time.sleep(0.5) # Wait for the response
    response = ser.read_all().decode() # Read and decode the response
    if response.startswith('+'): # Check if the response is valid
        parts = response.split('=') # Split the response at the '=' character

        return parts[1].strip() # Return the address (after '='), stripped of any whitespace
def set_parameter(ser,SF, BW, CR, Power ):
    command = f'AT+PARAMETER={SF},{BW},{CR},{Power}'
    ser.write((command + '\r\n').encode())
    time.sleep(0.5)
    ser.write(('AT+PARAMETER?' + '\r\n').encode())
    time.sleep(0.5)
    response = ser.read_all().decode()
    if response.startswith('+'): # Check if the response is valid
        parts = response.split('=')
        print(response)

def read_command(ser):
    try:
        end_time = time.time() + 30
        while time.time() < end_time: #30 seconds wait time
            string = ser.readline()
            c = string.decode("utf-8")
            if(c != ""):
                if c.startswith('+RCV='):
                    parts = c.split(',')
                    sender_id = parts[0].split('=')[1]
                    message_length = parts[1]
                    message_payload = parts[2]
                    rssi = parts[3]
                    snr = parts[4]
                    logging.info("Signal Strength: " + rssi)
                    return message_payload

    return "No input"

```

```
except Exception as e:
    logging.info("Error Read command")
```

9.2.3 Connect_to_Vehicle.py

```
""" .....
-- Engineer: Andres Flores
-- Description: establish a MAVLink connection to a drone via
-- a specified serial port and baud rate using the pymavlink library
..... """
from pymavlink import mavutil
# Connect to the vehicle on the specified serial port and baud rate
def connect_to_vehicle():
    try:
        print("Connecting to vehicle on /dev/ttyAMA0 with baud rate 57600...")
        master = mavutil.mavlink_connection('/dev/ttyAMA0', baud=57600)
        print("Waiting for heartbeat...")
        master.wait_heartbeat()
        print("Heartbeat received")
        return master
    except Exception as e:
        print(f'Failed to connect: {e}')
        return None
```

9.2.4 Set_mode.py

```
""" .....
-- Engineer: Andres Flores
-- Description: function changes the drone's mode to the
-- specified mode and confirms the change by waiting for
-- an acknowledgment from the system
..... """

from pymavlink import mavutil
import time
# Function to change the mode
def set_mode(master, mode):
    if mode not in master.mode_mapping():
        return False
```

```

mode_id = master.mode_mapping()[mode]
master.set_mode(mode_id)

# Confirm the mode change
ack = False
while not ack:
    msg = master.recv_match(type='COMMAND_ACK', blocking = True, timeout = 5)
    if msg and msg.command == mavutil.mavlink.MAV_CMD_DO_SET_MODE:
        if msg.result == mavutil.mavlink.MAV_RESULT_ACCEPTED:
            ack = True
        else:
            return False
    time.sleep(1)
return True

```

9.2.5 Check_Pre_Arm.py

```

""" .....
-- Engineer: Andres Flores
-- Description: function checks for a valid GPS lock or position
-- information from the vehicle's telemetry before allowing the
-- pre-arm sequence to proceed
..... """

from pymavlink import mavutil
# Function to check pre-arm status
def check_pre_arm(master):
    # Add specific checks here based on your setup
    print("Checking pre-arm status...")

    gps_lock = False
    while not gps_lock:
        print("Waiting ")
        msg = master.recv_match(blocking=True, timeout = 10)
        if msg:
            print(f'Received message: {msg.get_type()}')
            if msg.get_type() == 'GPS_RAW_INT' and msg.fix_type >= 3:
                gps_lock = True
                print("GPS lock acquired")

```

```

        elif msg.get_type() == 'GLOBAL_POSITION_INT':
            gps_lock = True
            print("Position information received")
            gps_lock = True
        else:
            print("No message received, retrying ...")
            time.sleep(1)

    print("Pre-arm check passed")
    return True

```

9.2.6 Arm_Drone.py

```

""" .....
-- Engineer: Andres Flores
-- Description: functions to arm a drone and check its armed
-- status using the MAVLink protocol via the pymavlink library
..... """

from pymavlink import mavutil
from connect_to_vehicle import connect_to_vehicle
import time

def arm_drone(master):
    # Send a command to arm the drone
    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0,
        1, # param1 (1 to arm, 0 to disarm)
        21196, 0, 0, 0, 0, 0
    )
    time.sleep(5)

def is_armed(master):
    # Request the current system status
    master.mav.command_long_send(
        master.target_system,
        master.target_component,

```

```

        mavutil.mavlink.MAV_CMD_REQUEST_MESSAGE,
        0,
        mavutil.mavlink.MAVLINK_MSG_ID_HEARTBEAT,
        0, 0, 0, 0, 0, 0
    )

    time.sleep(1)
    heartbeat = master.recv_match(type='HEARTBEAT', blocking=True)
    return heartbeat.base_mode & mavutil.mavlink.MAV_MODE_FLAG_SAFETY_ARMED !=
0

```

9.2.7 Disarm_Drone.py

```

""" .....
-- Engineer: Andres Flores
-- Description: functions to disarm drone
..... """
from pymavlink import mavutil
def disarm_drone(master):
    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0,
        0, # param1 (0 to disarm, 1 to arm)
        0, 0, 0, 0, 0, 0
    )
    print("Disarming Drone")

```

9.2.8 Takeoff.py

```

""" .....
-- Engineer: Andres Flores
-- Description: function to command a drone to ascend to a
-- specified altitude using the MAVLink protocol. It sets the
-- flight mode to GUIDED, sends a takeoff command
-- (MAV_CMD_NAV_TAKEOFF), and monitors altitude through
-- GLOBAL_POSITION_INT messages until the target is reached
-- or a timeout occurs.
..... """
from pymavlink import mavutil

```

```

from set_mode import set_mode
import time
def takeoff(master,altitude):

    if not set_mode(master,'GUIDED'):
        print("Failed to set GUIDED mode. Check GPS signal, pre-arm checks, and parameters.")
        return

    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_NAV_TAKEOFF,
        0,
        0, 0, 0, 0, 0, 0,
        altitude)

    master.mav.request_data_stream_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_DATA_STREAM_POSITION,
        1,
        1)
    start_time = time.time()
    timeout = 10
    while True:
        msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
        if msg:
            relative_altitude = msg.relative_alt/ 1000.0 # Altitude in meters
            if relative_altitude >= altitude - 0.1: # Allow a small margin
                break
            if time.time() - start_time > timeout:
                return
        time.sleep(1)

```

9.2.9 Set_Movement.py

```

""" .....
-- Engineer: Andres Flores
-- Description: set of drone control functions leveraging

```

```

-- MAVLink for precise and autonomous operations. It includes
-- obstacle detection and safe-distance management using LiDAR,
-- ensuring the drone halts and backs away when obstacles are
-- encountered (STOP_FLY and Fly_to_Safe_distance). For manual
-- movement, the script supports velocity and direction control
-- with dynamic adjustments (fly_movment). It facilitates waypoint
-- navigation by guiding the drone to specified GPS coordinates,
-- continuously monitoring for proximity hazards during
-- flight (fly_to_waypoint). Altitude adjustments can be performed
-- incrementally to reach desired heights safely (increse_alt).
-- Additionally, the script enables the drone to follow a circular
-- flight path, generating waypoints and navigating them in clockwise
-- or counterclockwise directions (fly_circle).
....."""

```

```

from pymavlink import mavutil
import logging
import math
import time
import serial
from lidar_distance import get_distance
from get_location import get_location
from travel_distance import distance_travel
from fly_forward import get_waypoint
from lora import send_command
from lidar_distance import get_current_distance

```

```

serial_port = '/dev/ttyUSB1'
baud_rate = 115200 # Default baud rate for RYLR998
GC_Address = 2
ser = serial.Serial(serial_port, baud_rate, timeout=1)
check_interval = 0.1
speed = 10 # drone speed 10 m/s
error = 0.5
safe_distance = 3

```

```

def STOP_FLY(master):

```

```

    master.mav.send(mavutil.mavlink.MAVLink_set_position_target_local_ned_message(10,
master.target_system,

```

```

        master.target_component,
        mavutil.mavlink.MAV_FRAME_BODY_OF
FSET_NED,

        int(0b11011111000 ),
        0, 0, 0,
        0, 0, 0,
        0, 0, 0,
        0, 0
    ))

msg = master.recv_match(type='COMMAND_ACK', blocking=True, timeout=5)

def Fly_to_Safe_distance(master):

    master.mav.send(mavutil.mavlink.MAVLink_set_position_target_local_ned_message(10,
master.target_system,

        master.target_component,
        mavutil.mavlink.MAV_FRAME_BODY_OF
FSET_NED,

        int(0b11011111000 ),
        0, 0, 0,
        0, 0, 0,
        -speed, 0, 0,
        0, 0
    ))

msg = master.recv_match(type='COMMAND_ACK', blocking=True, timeout=5)
duration = ( (abs(safe_distance) / speed) + error )
time.sleep(duration)

def fly_movment(master, x, y):

    # vx value for foward
    # vy value for right
    # vz value for down negative value for up
    # Bitmask to indicate to use Velocity only and Z position (positive is down) :
    0b110111000011
    master.mav.send(mavutil.mavlink.MAVLink_set_position_target_local_ned_message(10,
master.target_system,

        master.target_component,

```

```

FSET_NED,
                                mavutil.mavlink.MAV_FRAME_BODY_OF

                                int(0b110111111000 ),
                                x, y, 0,
                                0, 0 , 0,
                                0, 0, 0,
                                0, 0
                                ))

msg = master.recv_match(type='COMMAND_ACK', blocking=True, timeout=5)

if(x == 0):
    duration = ((abs(y) / speed ) + error )
elif(y == 0):
    duration = ( (abs(x) / speed) + error )
else:
    duration = (((abs(x) + abs(y)) / speed) + error)

end_time = time.time() + duration + error

while time.time() < end_time:
    try:
        distance = get_current_distance()
    except Exception as e:
        logging.info("Error trying to get current distance")
        send_command(ser, GC_Address, "INFO:Error trying to get current distance")

    if distance < safe_distance:
        STOP_FLY(master)
        Fly_to_Safe_distance(master)
        send_command(ser, GC_Address, "INFO:Obstacle Detected")
        logging.info("Obstacle Detected")
        break

def fly_to_waypoint(master, lat, lon, ALT):
    master.mav.send(mavutil.mavlink.MAVLink_set_position_target_global_int_message(10,
master.target_system,

```

```

master.target_component,
mavutil.mavlink.MAV_FRAME_GLOBAL_

RELATIVE_ALT_INT,

int(0b110111111000),
int(lat*1e7), int(lon* 1e7), ALT,
0, 0, 0,
0, 0, 0,
0, 0
))

tolerance=0.00001 # how close the drone needs to get to the target position before the loop
breaks
start_time = time.time() # Start time to track timeout duration
timeout = 120

while True:
    try:
        current_lat, current_lon, current_alt = get_location(master)
    except Exception as e:
        logging.info("Error trying to current location in fly_to_waypoint function")
        send_command.ser, GC_Address,"INFO:GET_LOCATION in fly point fail")
        print("ERROR TRYING TO GET CURRENT LOCATION")
    try:
        distance = get_current_distance()
    except Exception as e:
        logging.info("Error trying to get current distance")
        send_command.ser, GC_Address,"INFO>Error trying to get current distance")

lat_error = abs(abs(lat) - abs(current_lat))
lon_error = abs(abs(lon) - abs(current_lon))
if time.time() - start_time < timeout:
    if distance < safe_distance:
        STOP_FLY(master)
        Fly_to_Safe_distance(master)
        send_command.ser, GC_Address,"INFO:Obstacle Detected")
        break
    if(lat_error < tolerance and lon_error < tolerance ):
        logging.info("Reach Target Position")
        send_command.ser, GC_Address,"INFO:Reach Target Position")
        break

```

```

        else:
            logging.info("Enroute to target Position")
    else:
        send_command(ser, GC_Address, "INFO:Fly Waypoint timeout")
        logging.info("Waypoint Timeout")
        break

    time.sleep(check_interval)

def increse_alt(master, alt):
    master.mav.send(mavutil.mavlink.MAVLink_set_position_target_local_ned_message(10,
master.target_system,
                                                                    master.target_component,
                                                                    mavutil.mavlink.MAV_FRAME_BODY_OF
FSET_NED,
                                                                    int(0b110111000000),
                                                                    0, 0, -alt,
                                                                    0, 0, 0,
                                                                    0, 0, 0,
                                                                    0, 0
                                                                    ))
    master.mav.request_data_stream_send(master.target_system,
                                                                    master.target_component,
                                                                    mavutil.mavlink.MAV_DATA_STREAM_POSITION,
                                                                    1,
                                                                    1)
    start_time = time.time()
    timeout = 10
    while True:
        msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
        if msg:
            relative_altitude = msg.relative_alt/ 1000.0 # Altitude in meters
            print(f'Current altitude: {relative_altitude}')
            if relative_altitude >= alt - 0.1: # Allow a small margin
                print(f'Reached target altitude of {alt} meters")
                break
            if time.time() - start_time > timeout:
                print("Timeout reached. Unable to reach target altitude.")

```

```

        return
    time.sleep(1)
def fly_circle(master,radius,altitude,dir):

    num_waypoint = 8
    waypoints_lat = []
    waypoints_lon = []
    angle = 0
    for i in range(num_waypoint):
        lat, lon = get_waypoint(master, radius, angle)
        waypoints_lat.append(lat)
        waypoints_lon.append(lon)
        angle = angle + (360/num_waypoint)
        logging.info("Circle Waypoints : %f, %f" % (lat, lon))
        message = "INFO:Calculate CIRCLE Waypoint " + str(i+1) + "/" + str(num_waypoint)
        send_command (ser, GC_Address, message)
    if(dir == 0):
        for i in range(num_waypoint): #clock-wise fly path
            lat = waypoints_lat[i]
            lon = waypoints_lon[i]
            fly_to_waypoint(master, lat, lon, altitude )
            logging.info("Point %f complete " % (i+1))
            message = "ACK:Circle Waypoint " + str(i+1) + "/" + str(num_waypoint)
            send_command(ser, GC_Address, message)
    else:
        for i in range(num_waypoint): #counterclock-wise fly path
            lat = waypoints_lat[num_waypoint - i - 1]
            lon = waypoints_lon[num_waypoint - i - 1]
            fly_to_waypoint(master, lat, lon, altitude )
            logging.info("Point %f complete " % (i+1))
            message = "ACK:Circle Waypoint " + str(i+1) + "/" + str(num_waypoint)
            send_command(ser, GC_Address, message)

```

9.2.10 Range_Test.py

```

""" .....
-- Engineer: Andres Flores
-- Description:script tests the communication range of a LoRa
-- system integrated with a drone using MAVLink for navigation.
-- It generates waypoints at set intervals along the desired

```

```

-- target distance and guides the drone to each point
....."""
from pymavlink import mavutil
import serial
import time
import logging
from connect_to_vehicle import connect_to_vehicle
from LoRa_Commands import send_command
from get_location import get_location, get_waypoint
from set_movment import fly_to_waypoint
from datetime import datetime

check_interval = 0.5

def rx_test(ser):
    end_time = time.time() + 5
    while time.time() < end_time: #5 seconds wait time
        string = ser.readline()
        c = string.decode("utf-8")
        if(c != ""):
            if c.startswith('+RCV='):
                parts = c.split(',')
                sender_id = parts[0].split('=')[1]
                message_length = parts[1]
                message_payload = parts[2]
                rssi = parts[3]
                snr = parts[4]
                logging.info("Signal Strength: " + rssi)

            return True

    return False

def tx_test(ser, GC_Address):
    send_command(ser, GC_Address, "TEST:LoRa_CT ")
    rx = rx_test(ser)
    print(rx)
    return rx

```

```

def test_lora_comm_range(master, ser, GC_Address, Target_distance, altitude, home_lat,
home_lon):
    distance = 0
    i = 0
    num_waypoint = 5
    waypoints_lat = []
    waypoints_lon = []
    angle = 0
    signal_not_response = 0
    intervals = Target_distance/num_waypoint
    print(num_waypoint, intervals)

    logging.info("Total Distance for LoRa Comm Test: %f" % Target_distance)
    for n in range(num_waypoint):
        print("Point " , n)
        lat, lon = get_waypoint(master, (n+1)*intervals, angle)
        waypoints_lat.append(lat)
        waypoints_lon.append(lon)
        logging.info("Waypoints : %f, %f" % (lat, lon))
        message = "INFO:Calculate Waypoint " + str(n+1) + "/" + str(num_waypoint)
        send_command (ser, GC_Address, message)
    send_command (ser, GC_Address, "INFO:Start Test")
    while distance < Target_distance:
        rx = tx_test(ser,GC_Address)
        if rx:
            if i < num_waypoint:
                lat = waypoints_lat[i]
                lon = waypoints_lon[i]
                fly_to_waypoint(master, lat, lon, altitude)
                logging.info("Point %f complete " % (i+1))
                message = "ACK:Waypoint " + str(i+1) + "/" + str(num_waypoint)
                send_command(ser, GC_Address, message)
                i += 1
            else:
                logging.info("All waypoints complete")
                send_command(ser, GC_Address, "INFO:ALL Waypoint Complete")
                break

        else:
            logging.info("Singal Loss")

```

```

        signal_not_response += 1
        if(signal_not_response == 1 ):
            return False
        time.sleep(1)
        distance += intervals

    send_command(ser, GC_Address, "ACK:Flying back Home")
    fly_to_waypoint(master, home_lat, home_lon, altitude )

    logging.info("Test Past of %f meters" % distance)
    return True

```

9.2.11 Get_Location.py

```

""" .....
-- Engineer: Andres Flores
-- Description:script retrieves the GPS location of a drone
-- using MAVLink commands while monitoring the health of GPS data.
-- If valid GPS data is unavailable after multiple attempts, it
-- initiates an emergency landing and logs the issue, ensuring
-- safety during operation also calculate new coordinate waypoints
..... """
from pymavlink import mavutil
from LoRa_Commands import send_command
import serial
from land import land
import logging
import math

num_try = 0
max_try = 10

GC_Address = 2
serial_port = '/dev/ttyUSB0'
baud_rate = 115200 # Default baud rate for RYLR998
ser = serial.Serial(serial_port, baud_rate, timeout=1)

def get_location(master):
    master.mav.command_long_send(
        master.target_system,

```

```

        master.target_component,
        mavutil.mavlink.MAV_CMD_SET_MESSAGE_INTERVAL,
        0,
        mavutil.mavlink.MAVLINK_MSG_ID_GPS_RAW_INT, 1000000,
        0,0,0,0,0,0
    )
while True:

    msg = master.recv_match(type = 'GPS_RAW_INT', blocking = True, timeout = 1)

    if msg is not None:
        lat = msg.lat/1e7
        lon = msg.lon/1e7
        alt = msg.alt/1e3
        if(lat == 0.0 or lon == 0.0 or alt == 0.0):
            logging.info("INFO:GPS Bad Health EMERGENCY Landing")
            send_command.ser, GC_Address, "INFO:GPS Bad Health")
            send_command.ser, GC_Address, "INFO:EMERGENCY Landing")
            land(master)

        else:
            return lat, lon, alt

    elif(num_try < max_try):
        logging.info("INFO.WAITING FOR GPS")
        send_command.ser, GC_Address, "INFO:Waiting For Location" )
        num_try += 1
    else:
        logging.info("INFO:GPS NOT Responding EMERGENCY Landing")
        send_command.ser, GC_Address, "INFO:GPS NOT Responding")
        send_command.ser, GC_Address, "INFO:EMERGENCY Landing")

def get_waypoint(master, distance, angle):
    lat,lon, alt = get_location(master)
    Earth_R = 6378137.0

    lat = math.radians(lat)
    lon = math.radians(lon)
    offset = distance/Earth_R
    New_lat = lat + (offset * math.cos(angle))

```

```
New_lon = lon + (offset * (math.sin(angle)/math.cos(lat)))
```

```
New_lat = math.degrees(New_lat)
```

```
New_lon = math.degrees(New_lon)
```

```
return New_lat, New_lon
```

9.2.12 Drone_Menu.py

```
""" .....
-- Engineer: Andres Flores
-- Description: sends a menu of flight path options to a drone
-- via LoRa communication, allowing the operator to choose
..... """
from lora import send_command, read_command
import serial
def send_drone_flypath_menu(ser,GC_Address):
    send_command(ser, GC_Address, "INFO:1 = FLY Movement")
    send_command(ser, GC_Address, "MENU:2 = WAYPOINT")
    send_command(ser, GC_Address, "MENU:3 = HOVER")
    send_command(ser, GC_Address, "MENU:4 = Circle")
    send_command(ser, GC_Address, "MENU:5 = Return Home")
    send_command(ser, GC_Address, "MENU:6 = RETURN HOME AND LAND")
    send_command(ser, GC_Address, "MENU:7 = LAND")
    send_command(ser, GC_Address, "INPUT:8 = LoRa Com Test ")
    drone_command = read_command(ser)
    return drone_command
```

9.2.13 Lidar_Distance.py

```
""" .....
-- Engineer: Andres Flores
-- Description: script interfaces with a TF-Luna LiDAR sensor
-- to measure and calculate distances by reading data via a
-- serial connection, while allowing configuration of the
-- sample rate. It provides functionality to continuously
-- collect distance samples, average them, and return the
-- current distance in a thread-safe manner
..... """
import serial
```

```

import time
from threading import Lock

ser = None
current_distance = None
distance_lock = Lock()

# Open serial port
try:
    ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
except serial.SerialException as e:
    print(f'SerialException: {e}')
    exit()

# Function to send a command to set the sample rate
def set_sample_rate(rate):
    if rate < 1 or rate > 250:
        print("Sample rate must be between 1Hz and 250Hz.")
        return

    if ser.is_open:
        # Construct the command to set the sample rate
        command = bytearray([0x5A, 0x06, 0x03, rate, 0, 0, 0, 0])

        # Calculate the checksum
        checksum = sum(command) % 256
        command[-1] = checksum

        # Send the command
        ser.write(command)
    else:
        print("Serial port is not open.")

# Function to read and decode data from TF-Luna
def read_lidar_data():
    ser.flushInput() # Clear any residual data
    data = ser.read(9)

    # Check if the data frame is complete
    if len(data) == 9:

```

```

# Check for valid data frame starting with 0x59 0x59
if data[0] == 0x59 and data[1] == 0x59:
    # Extract the distance value (2 bytes: data[2] and data[3])
    distance = data[2] + (data[3] << 8)
    return distance * 0.01 # Convert to meters

else:
    print("Incomplete data frame received")
    return None

def avg_distance(num_samples):
    distances = [] # Initialize the list to store distances

    for _ in range(num_samples):
        dist = read_lidar_data() # Call the function to get a distance measurement
        if dist is not None: # Check if the measurement is valid
            distances.append(dist)

    if len(distances) == 0:
        return -1 # Return -1 if no valid distances were collected
    average_distance = sum(distances) / len(distances)

    return round(average_distance, 2)

def get_distance():
    global current_distance
    num_sample = 100
    set_sample_rate(250)
    while True:
        distance = avg_distance(num_sample)
        if(distance < 0.25): #lidar range is from 0.2 - 8.0 meters
            distance = 8
        with distance_lock:
            current_distance = distance

def get_current_distance():
    with distance_lock:

```

```
return current_distance
```

9.2.14 Land.py

```
""".....
-- Engineer: Andres Flores
-- Description: function initiates the drone's landing procedure
-- by setting the mode to "LAND" and sending the appropriate
-- MAVLink command, while continuously monitoring the altitude
-- until the drone reaches a safe landing threshold
....."""

from pymavlink import mavutil
from set_mode import set_mode
import time
#Function to land
def land(master):
    if not set_mode(master,'LAND'):
        print("Failed to set LAND mode. Check GPS signal, pre-arm checks, and parameters.")
        return
    print("Landing Start")

    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_NAV_LAND,
        0,0, 0, 0, 0, 0, 0,0)

    master.mav.request_data_stream_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_DATA_STREAM_POSITION,
        1,
        1)
    while True:
        msg = master.recv_match(type='GLOBAL_POSITION_INT', blocking=True)
        if msg:
            relative_altitude = msg.relative_alt/ 1000.0 # Altitude in meters
            print(f'Current altitude: {relative_altitude}')
```

```

        if relative_altitude < 0.1: # Allow a small margin
            print("Drone has land ")
            break
    time.sleep(1)

```

9.2.15 Signal_Test.py

```

""" .....
-- Engineer: Andres Flores
-- Description: script performs a signal connection test for
-- a drone using LoRa communication, with a thread that monitors
-- the signal and triggers a safe landing if the connection is lost
..... """
from pymavlink import mavutil
import time
import serial
import sys
from connect_to_vehicle import connect_to_vehicle
from lora import send_command
from set_movment import fly_to_waypoint
from land import land
from get_location import get_location
import threading
from lora import send_command, get_address, get_network, read_command, set_parameter

thread_stop_event = threading.Event()

def rx_test(ser):
    end_time = time.time() + 1
    while time.time() < end_time: #1 seconds wait time
        string = ser.readline()
        c = string.decode("utf-8")
        if c != "" and c.startswith('+RCV='):
            parts = c.split(',')
            sender_id = parts[0].split('=')[1]
            message_length = parts[1]
            message_payload = parts[2]
            rssi = parts[3]
            snr = parts[4]
            return True

```

```

    return False

def tx_test(ser, GC_Address):
    send_command(ser, GC_Address, "TEST: ")
    rx = rx_test(ser)
    return rx

def signal_connection(master, ser, home_lat, home_lon, GC_Address, altitude):
    signal_loss = 0
    while not thread_stop_event.is_set():
        signal_connect = tx_test(ser, GC_Address)
        print(signal_connect)
        if(signal_connect == False):
            signal_loss += 1
            if ( signal_loss == 2):
                print("loss Connection")
                fly_to_waypoint(master, home_lat, home_lon, altitude )
                land(master)
                print("land safe")
                sys.exit()

        else:
            print("signal Connect")
            signal_loss = 0
            time.sleep(0.5)

def start_thread_singal_connection(master, ser, home_lat, home_lon, GC_Address, altitude):
    signal_thread = threading.Thread(target=signal_connection, args=(master, ser, home_lat,
home_lon, GC_Address, altitude))
    signal_thread.start()

def stop_signal_connection_thread():
    thread_stop_event.set()

```

9.2.16 If_number.py

```

""" .....
-- Engineer: Andres Flores
-- Description: functions check whether a given value can be converted to a float or an integer
..... """
def is_number_float(value):

```

```

try:
    float(value)
    return True
except ValueError:
    return False

```

```

def is_number_int(value):
    try:
        int(value)
        return True
    except ValueError:
        return False

```

9.2.17 Log_File.py

```

""".....
-- Engineer: Andres Flores
-- Description: function sets up a log file with a timestamped
-- filename and configures logging to record drone test information
....."""
from datetime import datetime
import logging

def setup_log_file():
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    log_filename = f"/home/pi/Drone_Test_Logs/drone_LoRa_log_{timestamp}.log"
    logging.basicConfig(filename=log_filename,
                        level=logging.INFO,
                        format='%(asctime)s - %(levelname)s - %(message)s',
                        filemode='w')
    logging.info("Start")

```

Work Cite

- [1] ArduPilot, "Copter commands in guided mode," [Online]. Available:
<https://ardupilot.org/dev/docs/copter-commands-in-guided-mode.html>.
- [2] Reyax Technology Co., Ltd., "RYLR998 UART Interface 868/915 MHz LoRa Antenna Transceiver Module Datasheet." [Online]. Available:
https://reyax.com/upload/products_download/download_file/RYLR998_EN.pdf
- [3] RYLR998, "LoRa® AT Command Guide," [Online]. Available:
https://reyax.com/upload/products_download/download_file/LoRa_AT_Command_RYL_R998_RYLR498_EN.pdf
- [4] Semtech, "What is LoRa?," [Online]. Available: <https://www.semtech.com/lora/what-is-lora>.
- [5] Cavli Wireless, "AT commands tutorial," [Online]. Available:
<https://www.cavliwireless.com/blog/nerdiest-of-things/an-introduction-to-cellular-at-commands.html#:~:text=AT%20Commands%20or%20'ATtention'%20Commands,send%20instructions%20on%20various%20functions>
- [6] Mokosmart. (n.d.). "What is LoRa Technology and How it Works". [Online]. Available:
[https://www.mokosmart.com/lora-technology/#:~:text=Chirp%20Spread%20Spectrum%20\(CSS\),modulated%20onto%20a%20carrier%20frequency](https://www.mokosmart.com/lora-technology/#:~:text=Chirp%20Spread%20Spectrum%20(CSS),modulated%20onto%20a%20carrier%20frequency).